h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Hochschule Darmstadt

– Fachbereich Informatik –

A Classic Thesis Style

Exposé der Abschlussarbeit zur Erlangung des akademischen Grades
Thema: Evaluation adaptiver Sampling-Strategien für Distributed Tracing in event-getriebenen Serverless-Architekturen Bachelor of Science (B. Sc.)

vorgelegt von

Aksel Kenanov

Matrikelnummer: 1118200

Referent	:	Prof. Dr. Peter Kling
Korreferent	:	Prof. Dr. Martin Stiemerling

Aksel Kenanov: *A Classic Thesis Style*, © 7. Dezember 2025

ERKLÄRUNG

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Darmstadt, 7. Dezember 2025

Aksel Kenanov

INHALTSVERZEICHNIS

I Thesis

1 Problemstellung 2
- 1.1 Grundlagen . . . 2
  - 1.1.1 Event Driven Architektur . . . 2
  - 1.1.2 Serverless Architektur . . . 2
  - 1.1.3 Distributed Tracing . . . 3
  - 1.1.4 Opentelemetry . . . 3
  - 1.1.5 Jaeger . . . 5
  - 1.1.6 Grafana k6 . . . 5

2 Erkentnisinteresse 6
- 2.1 Definition . . . 6
- 2.2 Auswahl der Tools . . . 6

3 Fragestellung 8

4 Ziele und Hypothesen 9
- 4.1 Ziele . . . 9
- 4.2 Hypothesen . . . 9

5 Theoriebezug 10

6 Stand der Forschung 11
- 6.1 Einleitung . . . 11
- 6.2 Relevante Studien . . . 11
- 6.3 Forschungslücken . . . 12

7 Methode 13
- 7.1 Systemarchitektur . . . 13
- 7.2 Sampling-Strategien . . . 13
  - 7.2.1 Tail-based Sampling . . . 13
  - 7.2.2 Rate Limit Sampling . . . 14
- 7.3 Experiment-Durchführung . . . 14
  - 7.3.1 Testszenarios und Lasterzeugung . . . 14
  - 7.3.2 Fehler-Injektion . . . 15
  - 7.3.3 Messdurchführung und Wiederholungen . . . 15
- 7.4 Metriken und Messgröße . . . 15

8 Material 17
- 8.1 Cloud-Infrastruktur . . . 17
- 8.2 Tracing-Stack . . . 17
- 8.3 Programmiersprachen und Testing-Tools . . . 17
- 8.4 Monitoring und Datenerfassung . . . 18

Literatur 19

ABBILDUNGSVERZEICHNIS

Abbildung 2.1 Architekturdiagramm 6
Abbildung 7.1 Architekturdiagramm 13

TABELLENVERZEICHNIS

LISTINGS

ABKÜRZUNGSVERZEICHNIS

Teil I

THESIS

1 PROBLEMSTELLUNG

In stark skalierenden, event-getriebenen Serverless-Architekturen stellt Distributed Tracing eine essentielle Observability-Technologie dar. Die vollständige Erfassung aller Traces führt jedoch zu inakzeptabel hohem Performance-Overhead. Statische Sampling-Strategien können entweder zu viel Overhead verursachen oder kritische Traces – insbesondere Fehlerfälle – verpassen. Diese Arbeit adressiert das Spannungsfeld zwischen Tracing-Vollständigkeit und System-Overhead durch die Untersuchung adaptiver Sampling-Ansätze mit kostenlosen Open-Source-Tools.

1.1 GRUNDLAGEN

1.1.1 *Event Driven Architektur*

Was ist Event Driven Architektur (Eventgetriebene Architektur)? Es gibt diverse Definitionen für den Begriff Ëvent Driven Architecture", jedoch implizieren alle die gleiche Bedeutung. Laut den Autoren der Amazon Web Services Dokumentation ist Eventgetriebene Architektur wie folgt erklärt:
Ëine ereignisgesteuerte Architektur nutzt Ereignisse, um entkoppelte Dienste auszulösen und zu kommunizieren, und ist in modernen Anwendungen mit Microservices üblich. Ein Ereignis ist eine Zustandsänderung oder ein Update, zum Beispiel wenn ein Artikel in einen Einkaufswagen auf einer E-Commerce-Website gelegt wird. Ereignisse können entweder den Zustand tragen (der gekaufte Artikel, sein Preis und eine Lieferadresse) oder Ereignisse können Identifikatoren sein (eine Benachrichtigung, dass eine Bestellung versandt wurde).".

1.1.2 *Serverless Architektur*

1.1.2.1 *Definition*

Laut den Autoren der Google-Cloud-Platform-Dokumentation ist eine Serverless Architektur wie folgt definiert:
SServerlose Architekturen sind darauf ausgelegt, Server und Serververwaltung von den Entwicklungsteams zu abstrahieren. „Serverlos“ bedeutet nicht, dass keine Server vorhanden sind. Stattdessen bezieht sich der Begriff auf die Gesamtentwicklungserfahrung. Aus Sicht eines Entwicklers schreiben Sie einfach den Code und führen ihn aus, ohne sich sonst Sorgen machen zu müssen. Sämtliche Bereitstellung, Hardwarewartung, Software- und Sicherheitsupdates für die Server und andere Serververwaltungsaufgaben verant-

wortet der Cloud-Anbieter. Darüber hinaus skalieren serverlose Architekturen automatisch entsprechend dem Traffic.".

1.1.2.2 *Vorteile der Serverless Architektur*

Einige wichtige Vorteile von Serverless Architekturen sind vor allem:

- Automatisierung
  Serverlose Lösungen vereinfachen die Verwaltung von Servern durch automatisierte Aufgaben.

- Skalierbarkeit
  Serverlose Lösungen skalieren je nach Traffic automatisch hoch und herunter, ohne dass Sie Feinabstimmungen oder andere manuelle Konfigurationen vornehmen müssen.

- Produktivität
  Mit serverlosem Computing können sich Entwickler auf das Schreiben von Code und das Optimieren der Geschäftslogik konzentrieren, anstatt Zeit für die Serververwaltung aufzuwenden. Entwickler können ihren Code bei Bedarf auch direkt in Ad-hoc-Testumgebungen bereitstellen.

1.1.3 *Distributed Tracing*

Distributed Tracing (verteiltes Tracing) ist eine Technologie zur Observability in verteilten Systemen [Zai25]. Beim verteilten Tracing werden Datenanforderungen beobachtet, während sie ein verteiltes System durchlaufen. Eine moderne Microservices-Architektur besteht oft aus mehreren kleinen unabhängigen Komponenten. Diese Komponenten kommunizieren ständig und tauschen Daten mithilfe von APIs aus, um komplexe Aufgaben zu erledigen. Mit verteiltem Tracing können Entwickler den Anforderungspfad über verschiedene Microservices hinweg verfolgen oder visuell verfolgen [Ama25]. Diese Sichtbarkeit hilft bei der Behebung von Fehlern oder bei der Behebung von Bugs und Leistungsproblemen.

1.1.4 *Opentelemetry*

1.1.4.1 *Definition*

OpenTelemetry ist ein Open-Source-Standard und -Framework zur Sammlung von Telemetriedaten (Logs, Metriken und Traces) aus Cloud-nativen Anwendungen [Ope25a]. Es bietet einheitliche APIs, SDKs und Tools, um die Daten zu instrumentieren, zu erfassen und zu exportieren, wodurch eine herstellerneutrale Überwachung und Analyse verteilter Systeme ermöglicht wird. Ziel ist es, die Leistung und das Verhalten von Systemen besser zu verstehen und Probleme schneller zu identifizieren und zu beheben. Das

einzige, das diesem Framework fehlt ist eine Backend-Software die, die exportierten Daten visualisieren kann. Es gibt jedoch einige Backend-Softwares für die Speicherung und Visualisierung von exportierten Daten wie z. B. Jaeger, Zipkin, Grafana uvm.

1.1.4.2 *Sampling-Strategien von Opentelemetry-Framework*

Sampling ist der Prozess, bei dem entweder stichprobenartig oder regelbasiert nur eine bestimmte Anzahl an Telemetriedaten instrumentiert, erfasst und exportiert werden [Ope25b]. Dieser Ansatz wird häufig in Software mit sehr hoher Netzlast eingesetzt, um den Overhead zu vermeiden, der durch Instrumentierung, Erfassung und Export verursacht wird. Opentelemetry bietet zwei Sampling-Strategien an:

- Head Sampling
  Head Sampling ist eine Sampling-Technik, bei der die Entscheidung, ob ein Trace aufgezeichnet wird, so früh wie möglich getroffen wird. Die Entscheidung, einen Span oder Trace zu erfassen oder zu verwerfen, erfolgt dabei nicht durch die Betrachtung des gesamten Traces.
  Das gebräuchlichste Verfahren des Head Sampling ist das Consistent Probability Sampling, auch Deterministisches Sampling genannt. In diesem Fall wird die Entscheidung anhand der Trace-ID und des gewünschten Prozentsatzes der zu erfassenden Traces getroffen. Dadurch wird sichergestellt, dass vollständige Traces erfasst werden – ohne fehlende Spans – und zwar mit einer konstanten Rate, beispielsweise 5 Prozent aller Traces. Die Vorteile des Head Sampling sind:
  Einfach zu verstehen, Einfach zu konfigurieren, Effizient, Kann an jedem Punkt der Trace-Sammlungspipeline durchgeführt werden.

- Tail Sampling
  Tail Sampling ist ein Verfahren, bei dem die Entscheidung, ob ein Trace (Ablauf) aufgezeichnet wird, auf Grundlage aller oder der meisten Spans innerhalb des Traces getroffen wird. Tail Sampling bietet die Möglichkeit, Traces anhand spezifischer Kriterien auszuwählen, die aus verschiedenen Teilen eines Traces abgeleitet werden – etwas, das mit Head Sampling nicht möglich ist. Einige Beispiele, wie Tail Sampling eingesetzt werden kann:
  Immer Traces aufzeichnen, die einen Fehler enthalten, Traces basierend auf der gesamten Latenz aufzeichnen, Traces aufzeichnen, die bestimmte Attribute oder Werte in einem oder mehreren Spans enthalten; zum Beispiel mehr Traces von einem neu bereitgestellten Service erfassen, Unterschiedliche Sampling-Raten auf Traces anwenden, abhängig von bestimmten Kriterien, etwa wenn Traces nur von Services mit geringem Datenvolumen stammen im Vergleich zu Traces von Services mit hohem Datenvolumen.

1.1.5 *Jaeger*

1.1.5.1 *Definition*

Jaeger ist eine Open-Source-Plattform für verteiltes Tracing [Jae25b], die zur Überwachung und Fehlerbehebung komplexer Microservices eingesetzt wird. Sie hilft Entwicklern dabei, Leistungsengpässe zu identifizieren, Abhängigkeiten zwischen Services zu analysieren und den Ablauf von Anfragen in einem verteilten System nachzuverfolgen, um die Ursachen von Fehlern und Leistungsproblemen zu finden. Es fungiert als Backend-Software für bereits erfasste und exportierte Telemetriedaten. Es ergänzt somit Opentelemetry.

1.1.5.2 *Jaeger Collector und Jager Rate Limiting Sampling*

Der Jaeger Collector ist eine zentrale Backend-Komponente der Jaeger-Tracing-Plattform. Er empfängt die von Jaeger Agents oder direkt von Instrumentierungen gesendeten Traces/Spans, verarbeitet sie und leitet sie an ein Speicher-Backend z. B. Cassandra.

Rate Limiting Sampler (sampler.type=ratelimiting) [Jae25a] Dieser Sampler sorgt dafür, dass nur eine bestimmte Anzahl von Traces pro Sekunde aufgenommen wird. Er verwendet dazu das Prinzip eines Leaky Bucket Rate Limiters: Man kann sich das wie einen Eimer vorstellen, der mit einer konstanten Geschwindigkeit „ausläuft“. Neue Traces dürfen nur aufgenommen werden, solange der Eimer nicht „überläuft“. Dadurch wird eine gleichmäßige Rate garantiert, unabhängig davon, wie viele Requests tatsächlich eingehen.

1.1.6 *Grafana k6*

Grafana k6 ist ein Open-Source-Tool für Lasttests, das entwicklerfreundlich und erweiterbar ist. Mit k6 lassen sich Leistungsprobleme vermeiden und die Zuverlässigkeit proaktiv verbessern. Durch den Einsatz von k6 kann man die Zuverlässigkeit und Performance einer Anwendung und Infrastruktur testen. k6 unterstützt Engineering-Teams dabei, Fehler und Verstöße gegen SLOs (Service Level Objectives) zu verhindern und ermöglicht es ihnen, resilliente und leistungsstarke Anwendungen zu entwickeln, die skalierbar sind.
Last- und Performance-Tests:
k6 ist für minimalen Ressourcenverbrauch optimiert und wurde speziell für Hochlast-Performance-Tests wie Spike-, Stress- oder Soak-Tests entwickelt.

2 ERKENTNISINTERESSE

2.1 DEFINITION

Die Originalität dieser Arbeit liegt in der vergleichenden Evaluation zweier spezifischer adaptiver Sampling-Ansätze unter realen Serverless-Bedingungen. Während bestehende Arbeiten Sampling-Strategien allgemein untersuchen (siehe Kapitel 5. Theoriebezug und 6. Stand der Forschung), konzentriert sich diese Arbeit auf die praktische Implementierung und den Performance-Vergleich von Tail Sampling von Opentelemetry und Rate-Limiting in einer reproduzierbaren Testumgebung. Die eigenständige Leistung umfasst die Implementierung einer Serverless-Architektur, Konfiguration der Sampling-Strategien in OpenTelemetry und Jaeger-Backend empirische Analyse ihrer Effektivität, System-Overhead und eine Analyse der Trade-offs beider Sampling-Ansätze.

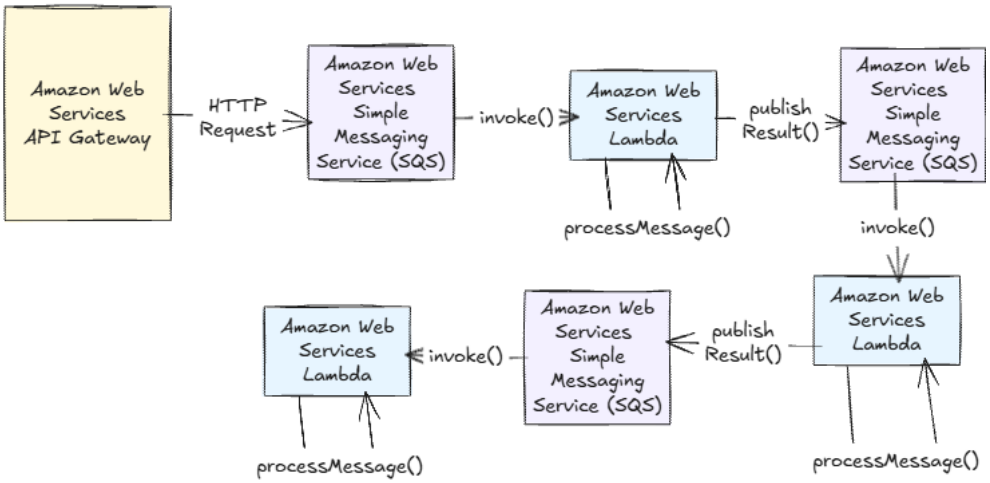

Abbildung 2.1: Architekturdiagramm

In der Arbeit werden anhand dieser Architektur die zwei genannten Sampling-Strategien (Tail Sampling und Rate Limiting Sampling) verglichen.

2.2 AUSWAHL DER TOOLS

Opentelemetry ist das Distributed-Tracing-Framework mit der verbreitetsten Unterstützung auf allen Cloud-Plattformen [Ope25a] - Google Cloud, Microsoft Azure und Amazon Web Services. Es ist eines der wenigen Frameworks, für die eine Integration für Cloud-Funktionen (AWS Lambda, Google Cloud Functions usw.) existiert [Ama25], da das Ziel dieser Arbeit adaptive Sampling-Ansätze für Serverless-Systeme zu untersuchen ist, eignet

sich Opentelemetry gut für den Zweck. Jaeger ist eine Open-Source Software [Jae25b], die sehr leichtgewichtig ist und sie ist komplementär zu Opentelemetry (Jaeger ergänzt einige Funktionalitäten von Opentelemetry). Außerderm hat Jager ein breites Feld an Unterstützung für die dauerhafte Aufbewahrung von Telemtriedaten (mit Speicher tools vie ElasticSearch, Cassandra usw.), was ein wichtiger Teild der Arbeit ist, es müssen nämlich empirische Analysen anhand von vorhandenen Telemetrie-Daten durchgeführt werden.

3 FRAGESTELLUNG

Leitfrage:
Wie beeinflussen unterschiedliche adaptive Sampling-Strategien (Tail Sampling vs. Adaptive Sampling) die Erfassungsrate kritischer Fehler-Traces und den Performance-Overhead in einer event-getriebenen Serverless-Architektur?
Wie es in einigen Arbeiten bewiesen wurde (siehe Kapitel 7. Stand der Forschung) [Kar24; EWL23] verursacht die Instrumentierung mit Telemetrie-Tools einen signifikanten System-Ovehead. Daher ist es wichtig, verschiedene Sampling-Ansätze zu vergleichen und daraus Schlüsse ziehen im Bezug auf Trade-Offs, verursachter Overhead, Trace-Completeness und Effektivität beim Erkennen der Fehler-Traces um konkrete Empfehlungen für die Praxis ableiten zu können - welche Strategie eignet sich am besten für einen Anwendungsfall? Unterfragen:
1. Welche Strategie erzielt die höchste Erfassungsrate kritischer Traces bei vergleichbarem Performance-Budget?
Die Arbeit [Kar24] legt empirische Evidenz vor, die besagt, dass Tail Sampling bei einem unbegrenzten Performance-Budeget (d. h. unbegrenzte Ressourcen für das Tracing) eine deutlich höhere Erfassungsrate von kritischen Traces hat, doch die Effektivität dieser Ansätze und deren Vergleich bleibt unter Begrenzung des Performance-Budgets unerforscht 2. Wie reagieren die verschiedenen Strategien auf Lastspitzen mit erhöhter Fehlerrate?
Obwohl es bereits Arbeiten gibt, die verschiedene Sampling-Strategien für Distributed Tracing detailliert untersucht haben (siehe Kapitel 7. [EWL23; Che+24]), bleibt das Verhalten der Strategien unter Lastspitzen mit erhöhter Fehlerrate unerforscht. 3. Welche Trade-offs ergeben sich zwischen Erfassungsgenauigkeit und Systemoverhead?
Diese Frage wurde anhand von empirischen Beweisen für Head Based und Tail Based erforscht [Kar24] (siehe Kapitel 7.), jedoch bleibt Frage für den Vergleich zwischen Tail Sampling von Opentelemetry und Adaptive Sampling von Jaeger offen.

4 ZIELE UND HYPOTHESEN

4.1 ZIELE

Ziele dieser Arbeit sind vor allem die Evaluation adaptiver Sampling-Strategien für Serverless-Anwendungen [EWL23]:

- Implementierung und Vergleich von Tail Sampling und Adaptive Sampling mit OpenTelemetry Collector und Jaeger [Ope25a; Jae25b].
- Quantifizierung des Trade-offs zwischen Trace-Erfassung und Performance-Overhead unter Betracht von verschiedenen Metriken [Kar24].
- Ableitung von Empfehlungen für den praktischen Einsatz in Serverless-Umgebungen für verschiedene Anwendungsfälle [NN25].

4.2 HYPOTHESEN

Zu untersuchende Hypothesen der Arbeit sind:

- H1: Tail-based Sampling erfasst deutlich mehr Fehler-Traces als Adaptive Rate Limiting bei vergleichbarem Performance-Overhead [Kar24].
- H2: Adaptive Rate Limiting zeigt stabileren Ressourcenverbrauch unter stark schwankenden Lastbedingungen [Jae25a].
- H3: Beide Strategien erfassen deutlich mehr Fehler-Traces als eine statische Sampling-Strategie z. B. 10 Prozent static sampling.

5 THEORIEBEZUG

Diese Arbeit stützt sich auf folgende theoretische Grundlagen:

- OpenTelemetry Sampling-Spezifikation [Ope25b]
- Jaeger Adaptive Sampling [Jae25a]
- Karkan, T. M. "Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure-[Kar24] (2024)

In der Opentelemetry-Dokumentation [Ope25a] und Jaeger-Dokumentation [Jae25b] sind die verschiedenen Sampling-Methoden zusammen mit Anleitungen wie sie konfiguriert werden können ausführlich Dokumentiert. Das sind zentrale Bezugspunkte für diese Arbeit, da die Arbeit den Vergleich dieser zwei Sampling-Strategien anstrebt.
Ein weiterer sehr wichtiger Bezugspunkt für diese Arbeit ist die Arbeit Karkan, T. M. "Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure-[Kar24] (2024). Die wissenschaftliche Arbeit vom Herrn Karakan dient als Stützpunkt für einige Aspekte der Messungen im Rahmen meiner eigenen Arbeit. Darunter sind vor allem Performance-Metriken wie:

- Server Response Time per Request - Die Runtime für die Abarbeitung einer User Requests durch das System.
- Server Response Time for all Requests - Die Runtime für die Abarbeitung aller User Requests durch das System.
- Total Memory Usage per Request - Der Speicherverbrauch für eine Request.
- Total Errors Detected - Anzahl der erfolgreich erfassten Fehler-Traces (pro Sampling-Strategie).

6 STAND DER FORSCHUNG

In diesem Kapitel wird der aktuelle Stand der Forschung zu dem in dieser Arbeit behandelten Thema dargestellt. Es werden relevante Studien vorgestellt, die als Grundlage für die weitere Untersuchung dienen.

6.1 EINLEITUNG

Der Stand der Forschung ist ein wesentlicher Bestandteil jeder wissenschaftlichen Arbeit. Er bietet einen Überblick über die bisherigen Erkenntnisse und zeigt auf, welche Lücken in der Forschung bestehen. In diesem Kapitel werden die wichtigsten Beiträge zum Thema zusammengefasst.

6.2 RELEVANTE STUDIEN

Die Forschung im Bereich Distributed Tracing und dessen Overhead-Auswirkungen ist von großer Bedeutung für die Entwicklung effizienter Cloud- und Serverless-Anwendungen. Drei zentrale Studien prägen den aktuellen Forschungsstand:

Sampling-Strategien und Performance-Overhead in Cloud-Infrastrukturen

Karkan [Kar24] untersucht in seiner Arbeit den Performance-Overhead verschiedener OpenTelemetry-Sampling-Methoden in Cloud-Infrastrukturen. Die Studie vergleicht drei unterschiedliche Sampling-Strategien: Head-based Sampling, Tail-based Sampling und eine Mischung aus beiden. Durch Experimente in Kubernetes-Umgebungen mit zwei und vier Knoten konnte gezeigt werden, dass Tail-based Sampling zwar die höchste Detektionsrate für Fehler bietet (71,33% CPU-, 23,7% Memory- und 5,6% Netzwerk-Overhead), aber auch den größten Overhead verursacht. Head-based Sampling zeichnet sich durch den geringsten Overhead aus, kann aber potenziell kritische Fehler übersehen. Diese Erkenntnisse verdeutlichen den Trade-off zwischen Observability-Vollständigkeit und Ressourceneffizienz.

Skalierbare und Streaming-basierte Sampling-Techniken

Chen et al. [Che+24] präsentieren mit Tracemesh eine innovative Lösung zur Adressierung der Skalierungsprobleme in der Trace-Verarbeitung. Tracemesh nutzt Locality-Sensitivity Hashing (LSH), um Traces in einen niedrigdimensionalen Raum zu projizieren und dabei ihre Ähnlichkeit zu bewahren. Das System passt die Sampling-Entscheidungen dynamisch durch evolving clustering an, um die Übersampling wiederkehrender Traces zu vermeiden.

Im Vergleich zu bestehenden Methoden demonstriert Tracemesh eine signifikante Verbesserung sowohl bei der Sampling-Genauigkeit als auch bei der Effizienz, was es zu einer vielversprechenden Lösung für produktive Systeme macht.

Overhead-Analyse von Distributed-Tracing-Tools in Serverless-Umgebungen

Eder et al. [EWL23] führen einen direkten Vergleich von drei weit verbreiteten Distributed-Tracing-Tools durch: Zipkin, OpenTelemetry und SkyWalking. Ihre Experimente basieren auf Benchmarks von AWS Lambda-Funktionsaufrufen und messen Runtime-Overhead, Memory-Verbrauch und Initialisierungsdauer. Die Ergebnisse zeigen, dass Zipkin mit durchschnittlich 10,73% Runtime-Overhead am effizientesten ist, während SkyWalking mit 50,67% Overhead am meisten Ressourcen benötigt. OpenTelemetry liegt mit 24,19% im mittleren Bereich. Diese Erkenntnisse sind besonders relevant für Serverless-Umgebungen, da dort die Kostenmodelle direkt an die Ressourcennutzung gekoppelt sind.

Schlussfolgerungen aus dem Forschungsstand

Die drei Studien verdeutlichen ein zentrales Spannungsfeld in der Distributed-Tracing-Technologie: Der Bedarf nach vollständiger Observability versus der Notwendigkeit zur Kostenoptimierung und Ressourceneffizienz. Während Karkan konzeptionelle Trade-offs zwischen Sampling-Strategien aufzeigt und Eder praktische Metriken für Tool-Auswahl bereitstellt, bietet Tracemesh einen technologischen Ansatz zur Reduzierung dieses Konflikts. Die Erkenntnisse dieser Arbeiten bilden die Grundlage für die vorliegende Arbeit.

6.3 FORSCHUNGSLÜCKEN

Aktuelle Stand der Forschung eine solide Grundlage für die vorliegende Arbeit bietet. Dennoch bestehen weiterhin Forschungslücken, die im weiteren Verlauf dieser Arbeit adressiert werden sollen. Insbesondere gibt es noch keine umfassende Arbeit die Rate Limiting Sampling und Tail Sampling vergleicht.

7 METHODE

In diesem Kapitel wird die methodische Vorgehensweise zur Evaluation verschiedener Sampling-Strategien für Distributed Tracing in Cloud-Umgebungen dargelegt. Das Kapitel behandelt die Auswahl und Konfiguration der Sampling-Strategien sowie das experimentelle Design zur Messung ihrer Performance-Charakteristiken.

7.1 SYSTEMARCHITEKTUR

Die Experimente werden in einer Serverless-Umgebung durchgeführt. Das Architekturdiagramm des Systems ist unten zu sehen

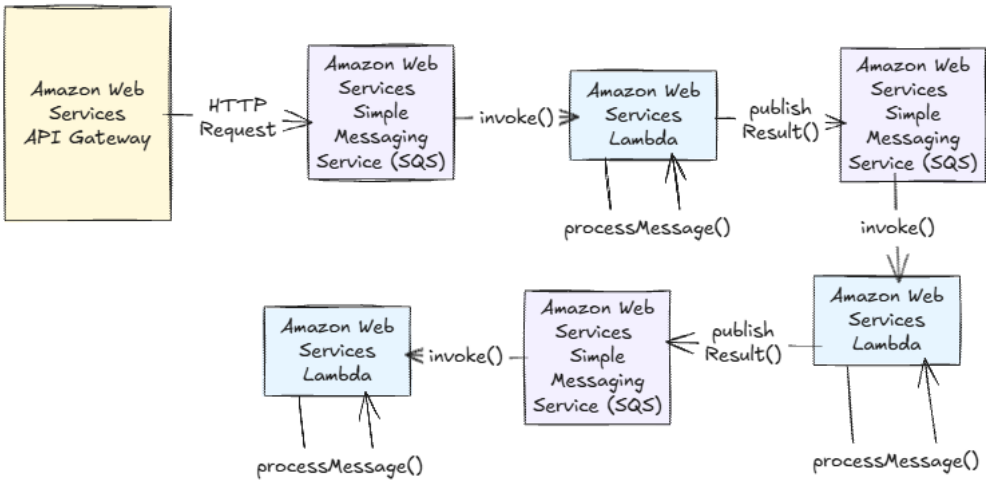

Abbildung 7.1: Architekturdiagramm

7.2 SAMPLING-STRATEGIEN

Um einen aussagekräftigen Vergleich zwischen verschiedenen Ansätzen zur Reduktion von Tracing-Overhead durchzuführen, werden drei Sampling-Strategien evaluiert: Tail-based Sampling, Rate Limit Sampling mit dynamischer Anpassung und statisches Sampling.

7.2.1 *Tail-based Sampling*

Tail-based Sampling basiert auf Eigenschaften der vollständigen Trace-Information und ermöglicht damit eine intelligentere Sampling-Entscheidung. In dieser Studie wird Tail-based Sampling mit den folgenden Parametern konfiguriert:

- **Fehler-basierte Regel:** Alle Traces mit Fehlern werden zu 100% erfasst, um sicherzustellen, dass fehlerhafte Anfragen vollständig für die Diagnose verfügbar sind.

7.2.2 *Rate Limit Sampling*

Rate Limit Sampling kontrolliert die Anzahl der Traces, die pro Zeiteinheit verarbeitet werden, und bietet dadurch eine präventive Methode zur Overhead-Limitierung.

7.2.2.1 *Dynamische Anpassung*

Die dynamische Variante des Rate Limit Sampling passt die maximale Trace-Rate in Echtzeit an die beobachteten Overhead-Metriken an. Konkret wird die Sampling-Rate basierend auf dem beim Tail-based Sampling gemessenen Overhead dynamisch kalibriert. Dies erlaubt es, dass beide Strategien unter vergleichbaren Ressourcen-Bedingungen arbeiten und somit deren intrinsische Unterschiede deutlich werden.

7.2.2.2 *Statisches Sampling*

Ergänzend wird auch ein statisches Rate Limit Sampling durchgeführt, bei dem die Sampling-Rate während des gesamten Experiments konstant bleibt. Diese Rate wird so kalibriert, dass sie durchschnittlich denselben Overhead wie das Tail-based Sampling verursacht. Das statische Sampling dient als Baseline-Verfahren zur Bewertung des Mehrwertes dynamischer Anpassungsmechanismen.

7.3 EXPERIMENT-DURCHFÜHRUNG

7.3.1 *Testszenarios und Lasterzeugung*

Die Experimente werden unter zwei verschiedenen Last-Profilen durchgeführt, um das Verhalten der Sampling-Strategien unter unterschiedlichen Bedingungen zu evaluieren:

- **Konstante Last:** Ein kontinuierlicher, gleichmäßiger Stream von Anfragen über die gesamte Testdauer. Dies simuliert typische Produktions-Workloads mit vorhersagbarem Durchsatz.
- **Burst-Szenarien:** Plötzliche Spitzen im Request-Volumen, gefolgt von Phasen normaler Last. Dies modelliert realistische Situationen wie Traffic-Spitzen, Marketing-Kampagnen oder unerwartete Lastanstiege.

Die Lasttests werden mit dem Open-Source-Load-Testing-Tool k6 durchgeführt. k6 ermöglicht die präzise Definition von komplexen Last-Profilen, die Simulation vieler virtueller Benutzer und die Erfassung detaillierter Performance-Metriken. Alternativ werden bei Bedarf spezialisierte Test-Skripte entwickelt,

die genau auf die Anforderungen der Experiment-Durchführung abgestimmt sind.

7.3.2 *Fehler-Injektion*

Um die Effektivität der verschiedenen Sampling-Strategien bei der Erfassung von Fehler-Traces zu evaluieren, werden während der Experimente gezielt Fehler injiziert:

- **Fehlerrate während normaler Phase:** Während Phasen konstanter Last wird eine Fehlerrate von 2% aufrechterhalten, um kontinuierlich Fehler-Szenarien zu generieren.
- **Erhöhte Fehlerrate in Burst-Phasen:** Während der Burst-Szenarien wird die Fehlerrate auf 10% erhöht, um realistische Situationen zu simulieren, in denen erhöhte Last auch zu erhöhten Fehlerraten führt.
- **Fehlerarten:** Die injizierten Fehler umfassen verschiedene Typen wie Timeouts, Application Errors und transiente Fehler, um eine realistische Fehler-Verteilung zu repräsentieren.

7.3.3 *Messdurchführung und Wiederholungen*

Für jede Sampling-Strategie werden insgesamt drei komplette Testläufe durchgeführt, um die Robustheit und Konsistenz der Ergebnisse zu gewährleisten:

- **Testlauf-Dauer:** Jeder einzelne Testlauf hat eine Dauer von mindestens 10 Minuten, um ausreichend Datenpunkte für statistisch signifikante Analysen zu erfassen.
- **Wiederholung:** Durch die Durchführung von drei Wiederholungen pro Strategie werden Anomalien und zufällige Schwankungen minimiert. Die Ergebnisse werden durch Mittelwertbildung und Berechnung von Standardabweichungen aggregiert.
- **Bereinigung zwischen Läufen:** Zwischen den einzelnen Testläufen erfolgt eine Bereinigung der Caches und des Trace-Speichers, um Cross-Kontamination zwischen den Experimenten auszuschließen.

7.4 METRIKEN UND MESSGRÖSSE

Die Evaluation der Sampling-Strategien erfolgt anhand folgender Schlüssel-Metriken:

- **Performance-Overhead:** Runtime, Memory-Verbrauch gemessen als Prozentuale Steigerung gegenüber einer Baseline ohne Tracing.
- **Fehler-Erfassungsrate:** Der Anteil der Traces mit Fehlern, die von jeder Sampling-Strategie erfasst werden, im Verhältnis zur Gesamt-Fehleranzahl.

- **Latenz der Trace-Verarbeitung:** Die Zeit zwischen dem Auftreten eines Events und dessen Verfügbarkeit im Tracing-Backend.
- **Kosten-Effizienz:** Das Verhältnis zwischen erfassten Fehler-Traces und dem verursachten Overhead, zur Bewertung der Ressourcen-Effizienz jeder Strategie.

8 MATERIAL

In diesem Kapitel werden die eingesetzten Tools, Technologien und Ressourcen beschrieben, die zur Durchführung der Experimente verwendet werden.

8.1 CLOUD-INFRASTRUKTUR

Die Experimente werden auf der Amazon Web Services (AWS) Free Tier durchgeführt. Dies ermöglicht eine kosteneffiziente Evaluation mit einem verfügbaren Budget von 200 Euro. Folgende AWS-Services werden genutzt:

- **AWS Lambda:** Serverless Compute-Plattform für die Ausführung der Test-Funktionen und Simulierung von Microservices.
- **Amazon SQS:** Message Queue Service zur Entkopplung von Services und Simulation von asynchronen Workflows.
- **Amazon CloudWatch:** Monitoring- und Logging-Plattform zur Erfassung von Metriken und Logs aus Lambda-Funktionen.

8.2 TRACING-STACK

Das Distributed Tracing wird mit OpenTelemetry und Jaeger implementiert:

- **OpenTelemetry Collector:** Empfängt, verarbeitet und exportiert Telemetrie-Daten (Traces, Metriken, Logs) von den Anwendungen. Der Collector führt die Sampling-Strategien durch und aggregiert die Telemetrie-Daten.
- **Jaeger Backend:** Open-Source-System für Distributed Tracing, das die Traces speichert und eine Web-UI zur Visualisierung und Analyse der Traces bereitstellt.
- **CloudWatch Metrics:** Zusätzlich werden OpenTelemetry-Metriken in CloudWatch exportiert, um Performance-Daten wie CPU, Memory und Netzwerk-Verbrauch zu erfassen.

8.3 PROGRAMMIERSPRACHEN UND TESTING-TOOLS

- **Programmiersprachen:** Die Test-Anwendungen werden in Python und/oder TypeScript implementiert, um realistische Szenarien zu modellieren und die Interoperabilität der OpenTelemetry-Instrumentierung zu demonstrieren.

- **k6:** Das Open-Source-Load-Testing-Tool k6 wird für die Lasterzeugung eingesetzt. Es ermöglicht die Definition komplexer Last-Profile und die Erfassung detaillierter Performance-Metriken während der Tests.
- **Custom Test-Skripte:** Zusätzlich können spezialisierte Test-Skripte in Python oder TypeScript entwickelt werden, um gezielt Fehler zu injizieren und spezifische Szenarien zu simulieren.

8.4 MONITORING UND DATENERFASSUNG

Die Überwachung und Analyse der Experiment-Ergebnisse erfolgt durch:

- **CloudWatch Metriken:** Erfassung von CPU, Memory, Request Count und Fehlerrate auf Lambda-Funktionen.
- **OpenTelemetry Metriken:** Custom Metriken zur Messung von Sampling-Raten, Trace-Latenz und Overhead-Charakteristiken.
- **Jaeger UI:** Visuelle Analyse und Debugging von Traces zur Überprüfung der Sampling-Effektivität.

9 VORLÄUFIGE GLIEDERUNG

In diesem Kapitel wird die vorläufige Gliederung der Arbeit dargestellt, um einen strukturierten Überblick über die geplanten Inhalte und deren Anordnung zu geben.

1. Einleitung und Motivation
2. Grundlagen von Distributed Tracing und Serverless-Architekturen
3. Stand der Forschung zu Sampling-Strategien und Distributed Tracing
4. Methodik: Experiment-Design und Implementierung
5. Ergebnisse: Performance-Metriken und Trace-Erfassung
6. Diskussion: Trade-off-Analyse und Strategie-Vergleich
7. Implikationen für die Praxis
8. Fazit und Ausblick

10 VORLÄUFIGER PROJEKT- UND ZEITPLAN

Das Projekt ist in vier Hauptphasen unterteilt, die jeweils spezifische Aufgaben und Meilensteine umfassen. Während allen Phasen wird der entsprechende Text zur Aufgabe verfasst.Der vorläufige Zeitplan für die Durchführung des Projekts ist wie folgt:

10.1 PHASE 1: VORBEREITUNG

- Vertiefende Literaturrecherche
- Test-Architektur designen und implementieren
- OpenTelemetry Basis-Integration

10.2 PHASE 2: IMPLEMENTIERUNG

- Sampling-Strategien im OpenTelemetry Collector konfigurieren
- Test-Setup und Automation finalisieren
- Pilot-Tests und Debugging

10.3 PHASE 3: EXPERIMENTE

- Durchführung aller Lasttests
- Datensammlung und -aufbereitung

10.4 PHASE 4: AUSWERTUNG & DOKUMENTATION

- Ausführliche Datenanalyse

LITERATUR

[Ama25] Amazon Web Services. *Implementing Distributed Tracing – AWS Distro for OpenTelemetry*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/otel/latest/adot/lambda-tracing.html.

[Che+24] Zhuangbin Chen, Zhihan Jiang, Yuxin Su, Michael R. Lyu und Zibin Zheng. "Tracemesh: Scalable and Streaming Sampling for Distributed Traces". In: *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. 2024, S. 54–65. DOI: 10.1109/CLOUD62652.2024.00016.

[EWL23] Christina Eder, Stefan Winzinger und Robin Lichtenthäler. "A Comparison of Distributed Tracing Tools in Serverless Applications". In: *2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 2023, S. 98–105. DOI: 10.1109/SOSE58276.2023.00018.

[Jae25a] Jaeger Project. *Jaeger Documentation: Adaptive Sampling*. Accessed: 2025-12-07. 2025. URL: https://www.jaegertracing.io/docs/latest/sampling/.

[Jae25b] Jaeger Project. *Jaeger Documentation*. Accessed: 2025-12-07. 2025. URL: https://www.jaegertracing.io/docs/.

[Kar24] T. M. Karkan. "Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure". In: (2024). Published 2024.

[NN25] Vorname Nachname und Vorname2 Nachname2. *Optimizing Distributed Tracing: Best practices for remaining within budget and capturing critical traces*. Datadog Architecture Center. Accessed: 2025-12-07. 2025. URL: https://www.datadoghq.com/architecture/optimizing-distributed-tracing-best-practices-for-remaining-within-budget-and-capturing-critical-traces/.

[Ope25a] OpenTelemetry Community. *OpenTelemetry Documentation*. Accessed: 2025-12-07. 2025. URL: https://opentelemetry.io/docs/.

[Ope25b] OpenTelemetry Community. *OpenTelemetry Official Documentation: Sampling Configuration*. Accessed: 2025-12-07. 2025. URL: https://opentelemetry.io/docs/concepts/sampling/.

[Zai25] Mohammad Zaigam. *Distributed Tracing Guide: Best Practices 2025*. Accessed: 2025-12-07. 2025. URL: https://www.apica.io/blog/what-is-distributed-tracing-how-it-works-and-best-practices.